PiNNTORP LLC

Online Gambling Platform

Phase 3: Testing Plan

CSCI 2040U - Software Design and Analysis

Adrian Ahmadi • Hayden Dunn • Eric Beaulne • Nikola Grujin

1. Introduction

1.1 Testing Overview

This document represents the structured test plan for the PiNNTORP application, an online browser-based gambling application using JavaScript (HTML, CSS, JS) for the front-end and Java HTTP/WebSocket for the back-end. The objective is to ensure all the delivered features are correct on their own, work well with the neighboring features without defects, and meet the end-to-end user stories described in the original stories. The test plan has three test levels.

1.2 System Components Under Test

- JS Core - state.js (global in-memory state), storage.js (persistence wrapper for localStorage).
- JS Games - slotGame.js, diceRoll.js, coinFlip.js, each with its own logic for generating random numbers and calculating profits for winning or losing.
- JS Features - friends.js (add, remove, list friends), stats.js (read, reset statistics), account.js (delete account), rec.js (game recommendations based on friend activities).
- JS Views - gameView.js, friendsView.js, statsView.js, recView.js, responsible for reflecting the application's state on the DOM.
- Java Back-end - LoginHandler (register, login, logout), FriendsHandler (add, remove, list, accept, decline friend requests), UserStore (in-memory user storage), SessionManager (session management), Database (JSON persistence on files).

1.3 Testing Frameworks & Tools

- Node.js Built-in Test Runner (node --test / node:assert) - for all JavaScript unit tests. Tests are already wired and run via run-tests.bat.
- Manual Browser Testing - for view-layer integration and system tests where DOM manipulation is needed.
- JUnit 5 - to be used for Java back-end unit and integration tests for LoginHandler, UserStore, SessionManager, and Database.
- Postman / curl - for manually sending HTTP requests to the Java server during backend integration tests.

1.4 Testing Views Applied

All three testing perspectives are used throughout the plan:

- Clear Box (CB) - used for pure logic functions written in JS and Java handlers where the entire source code is available and internal branching is verified.
- Translucent Box (TB) - used for integration tests where component boundaries are well understood but internal implementation details of other modules are not fully exercised.
- Opaque Box (OB) - used for system tests driven by the browser user interface where only inputs and outputs are available without any knowledge of internal state.

2. Unit Tests

Unit tests are used to test individual functions. All JavaScript tests use Node.js's built-in test runner (node --test) with mock localStorage and are automated. For Java unit tests, JUnit 5 is used. The ID for all tests is **UT-XX-TV**.

2.1 Friends Module - friends.js

Test ID	Method / Class	Input(s)	Expected Output(s)	Approach	Assignee
UT-01-CB	addFriend(name) friends.js	name = “ “ (blank/whitespace)	Returns false, friends list unchanged, localStorage not written	Automated (node --test)	Hayden
UT-02-CB	addFriend(name) friends.js	name = “not_a_real_user” (unknown user)	Returns false, alert(“User does not exist”) fired, friends list unchanged	Automated (node --test)	Hayden
UT-03-CB	addFriend(name) friends.js	name = “DUMMY_ALICE” (wrong case)	Returns false, case-sensitive lookup fails, list unchanged	Automated (node --test)	Hayden
UT-04-CB	addFriend(name) friends.js	name = “dummy_charlie” (valid, not yet friend)	Returns true, state.users.main_user.friends includes dummy_charlie, localStorage updated	Automated (node --test)	Hayden
UT-05-CB	removeFriend(name) friends.js	name = “dummy_alice” (existing friend)	friends list becomes [], localStorage persisted with empty array	Automated (node --test)	Hayden
UT-06-CB	removeFriend(name) friends.js	name = “dummy_charlie” (not in list)	State and localStorage unchanged, getFriends() still returns [“dummy_alice”]	Automated (node --test)	Hayden

2.2 Statistics Module - stats.js

Test ID	Method / Class	Input(s)	Expected Output(s)	Approach	Assignee
UT-07-CB	getStats() stats.js	State: 12 played, 8 wins, 4 losses, profit 65, balance 165	Returns { balance:165, wins:8, losses:4, gamesPlayed:12, profit:65, ratio:”2.00” }	Automated (node --test)	Eric
UT-08-CB	getStats() stats.js	State: 5 played, 5 wins, 0 losses (division-by-zero edge case)	Returns ratio = 5 (integer, not string, losses denominator avoided)	Automated (node --test)	Eric
UT-09-CB	resetStats() stats.js	State: any non-zero stats, friends list preserved	All numeric fields set to 0, balance reset to 100, friends list kept, localStorage updated	Automated (node --test)	Eric

2.3 Account Module - account.js (TDD RED - currently a stub only)

Test ID	Method / Class	Input(s)	Expected Output(s)	Approach	Assignee
UT-10-CB	deleteCurrentAccount() account.js	currentUser = “main_user”, users: main_user,	Returns “main_user”, state.users.main_user is undefined,	Automated (node --test)	Eric

Test ID	Method / Class	Input(s)	Expected Output(s)	Approach	Assignee
		dummy_alice, dummy_bob	state.currentUser is null		
UT-11-CB	deleteCurrentAccount() account.js	dummy_alice.friends = ["main_user"], dummy_bob.friends = ["main_user", "dummy_alice"]	After deletion: dummy_alice.friends = [], dummy_bob.friends = ["dummy_alice"] (main_user scrubbed from all lists)	Automated (node -- test)	Eric
UT-12-CB	deleteCurrentAccount() account.js	Existing localStorage state with main_user	localStorage updated: currentUser null, main_user key absent, remaining users' friend lists corrected	Automated (node -- test)	Eric

2.4 Game Modules - slotGame.js, diceRoll.js, coinFlip.js

Test ID	Method / Class	Input(s)	Expected Output(s)	Approach	Assignee
UT-13-CB	playSlotRound() slotGame.js	RNG seeded so all three values equal (e.g. 5,5,5) - 3-of-3 match	wins += 1, profit += 5 * 5 = 25, gamesPlayed += 1, saveState called	Automated (node -- test)	Nikola
UT-14-CB	playSlotRound() slotGame.js	RNG seeded so exactly two values equal (e.g. 3,3,7) - 2-of-3 match	wins += 1, profit += 10, gamesPlayed += 1	Automated (node -- test)	Nikola
UT-15-CB	playSlotRound() slotGame.js	RNG seeded so all three values differ (e.g. 1,4,6) - no match	losses += 1, profit -= 5, gamesPlayed += 1	Automated (node -- test)	Nikola
UT-16-CB	rollDice(guess) diceRoll.js	guess = 4, RNG returns 4 (correct)	wins += 1, profit += 10, gamesPlayed += 1, returns { guess:4, roll:4, won:true }	Automated (node -- test)	Nikola
UT-17-CB	rollDice(guess) diceRoll.js	guess = 2, RNG returns 5 (incorrect)	losses += 1, profit -= 5, gamesPlayed += 1, returns { guess:2, roll:5, won:false }	Automated (node -- test)	Nikola
UT-18-CB	flipCoin(guess) coinFlip.js	guess = "Heads", RNG returns >= 0.5 ("Heads")	wins += 1, balance += 10, profit += 10, gamesPlayed += 1, returns "Heads"	Automated (node -- test)	Nikola
UT-19-CB	flipCoin(guess) coinFlip.js	guess = "Heads", RNG returns < 0.5 ("Tails")	losses += 1, balance -= 5, profit -= 5, gamesPlayed += 1, returns "Tails"	Automated (node -- test)	Nikola

2.5 Recommendation Module - rec.js

Test ID	Method / Class	Input(s)	Expected Output(s)	Approach	Assignee
UT-20-CB	getRecommendedGames() rec.js	User has 0 favorite games, one friend whose favoriteGames = [“Slots”, “Dice Roll”]	Returns [{ game:”Slots”, friends:[“friend1”] }, { game:”Dice Roll”, friends:[“friend1”] }] (sorted by tally)	Automated (node -- test)	Adrian
UT-21-CB	getRecommendedGames() rec.js	User favoriteGames = [“Slots”], friend favoriteGames = [“Slots”, “Coin Flip”]	“Slots” is skipped (already in user favorites), returns [{ game:”Coin Flip”, friends:[...] }]	Automated (node -- test)	Adrian
UT-22-CB	getRecommendedGames() rec.js	Two friends both have “Slots” in favorites, one also has “Dice Roll”	“Slots” appears first (tally = 2), “Dice Roll” second (tally = 1), sorted correctly	Automated (node -- test)	Adrian
UT-23-CB	getRecommendedGames() rec.js	User has no friends (friends = [])	Returns empty array []	Automated (node -- test)	Adrian

2.6 Java Back-end - LoginHandler

Test ID	Method / Class	Input(s)	Expected Output(s)	Approach	Assignee
UT-24-CB	handle(exchange) LoginHandler	HTTP GET request to /login (wrong method)	Response code 405, no session created	Automated (JUnit 5)	Nikola
UT-25-TB	handle(exchange) LoginHandler	POST { action:”login”, username:”alice”, password:”pass” } - user exists, password correct	Response 200, JSON body contains sessionID and success:true	Automated (JUnit 5)	Nikola
UT-26-TB	handle(exchange) LoginHandler	POST { action:”login”, username:”alice”, password:”wrong” } - wrong password	Response 200, JSON body contains success:false, no sessionID field	Automated (JUnit 5)	Nikola
UT-27-CB	handle(exchange) LoginHandler	POST { action:”register”, username:”newuser”, password:”abc” }	Response 200, user registered in UserStore, sessionID returned	Automated (JUnit 5)	Nikola

3. Integration Tests

Integration tests evaluate the interactions between multiple components. The test ID format is **IT-XX-TV**.

3.1 JS Frontend - Game Logic → State → Storage Pipeline

Test ID	Components	Preconditions	Steps	Expected Result	Assignee
IT-01-CB	slotGame.js + state.js + storage.js	User state initialised, localStorage mock present	Call playSlotRound(), read state.users[currentUser], read localStorage key	State fields (wins/losses/profit/gamesPlayed) updated correctly AND localStorage JSON reflects the same values	Hayden
IT-02-CB	diceRoll.js + state.js + storage.js	User state initialised, mock localStorage present	Call rollDice(3), inspect state, inspect localStorage	Win or loss recorded correctly in both the state and the localStorage at the same time	Hayden
IT-03-CB	coinFlip.js + state.js + storage.js	User state initialised, mock localStorage present	Call flipCoin("Heads"), inspect balance, profit, wins/losses in state and storage	Balance updated along with the profit, with both the state and localStorage reflecting the correct values	Hayden

3.2 Recommendation → Friends → State Pipeline

Test ID	Components	Preconditions	Steps	Expected Result	Assignee
IT-04-CB	rec.js + friends.js + state.js	Two users exist, user A is friends with user B who has favoriteGames populated	Call getRecommendedGames() for user A, inspect result	Returns games from user B's favorites that are not in user A's favorites, tally correctly counts shared recommendations across multiple friends	Adrian
IT-05-CB	rec.js + friends.js + state.js	User's friends list starts empty, addFriend() is called to add friend with favorites	Add friend via addFriend(), then call getRecommendedGames()	The recommendation list is now non-empty after adding the friend, which means rec.js does read the live state and not a stale snapshot	Adrian

3.3 Statistics View → Stats Logic → State

Test ID	Components	Preconditions	Steps	Expected Result	Assignee
IT-06-TB	statsView.js + stats.js + state.js	Page loaded with a user in state, stats section visible in DOM	Play one slot round, check DOM stats panel values	DOM counters update without page refresh to reflect new gamesPlayed, wins/losses, profit	Eric

Test ID	Components	Preconditions	Steps	Expected Result	Assignee
IT-07-CB	stats.js + state.js + storage.js	State seeded with non-zero stats	Call resetStats(), call getStats(), check localStorage	getStats() returns all zeros; localStorage reflects new state; friends list is preserved in state and localStorage	Eric

3.4 Java Back-end - LoginHandler → UserStore → SessionManager

Test ID	Components	Preconditions	Steps	Expected Result	Assignee
IT-08-TB	LoginHandler + UserStore + SessionManager	Server running, no users registered	POST /login { action:"register", username:"alice", password:"pass" }, then POST /login { action:"login", username:"alice", password:"pass" }	New registration creates user in UserStore. New login returns valid sessionID recognized by SessionManager	Nikola
IT-09-TB	LoginHandler + UserStore + SessionManager	Server running, user "alice" registered	POST /login { action:"logout", sessionID:"<valid>" }	Session is removed from SessionManager; any future requests with this sessionID are rejected	Nikola

3.5 Java Back-end - FriendsHandler → UserStore → Session Auth

Test ID	Components	Preconditions	Steps	Expected Result	Assignee
IT-10-TB	FriendsHandler + UserStore + Session	Two registered users, both have valid sessions	POST /friends { action:"add", sessionID:"<alice>", playerID:<alice>, friendID:<bob> }, then list-requests for bob	Friend request is in the received list of bob, and alice has the request in the sent list	Nikola
IT-11-TB	FriendsHandler + UserStore + Session	alice has sent bob a request, bob has a valid session	POST /friends { action:"accept", sessionID:"<bob>", playerID:<bob>, friendID:<alice> }	Both alice and bob are in each other's friend lists, and the request is removed from the pending queues	Nikola
IT-12-CB	FriendsHandler + Session	Any user with a mismatched sessionID	POST /friends { action:"list-friends", sessionID:"WRONG", playerID:<id> }	Response 401 is given, and no friend data is leaked	Nikola

4. System Tests

System tests verify end-to-end workflows from the end-user's point of view in the browser. All system tests are executed manually using Opaque Box view unless otherwise mentioned. The format of test IDs is **ST-XX-TV**.

4.1 Gameplay Workflows

Test ID	Scenario	Preconditions	Steps	Expected Outcome	Assignee
ST-01-OB	Full Slot Machine session	App running at localhost:5500, user profile exists in localStorage	1) Open app 2) Navigate to Slots 3) Click Roll 5 times 4) Open Stats panel	Stats panel is displaying accurate information for cumulative games played, won, W/L ratio, and profit after all 5 spins. The information is retained even after page refresh.	Hayden
ST-02-OB	Full Dice Roll session	App running, user profile in localStorage	1) Navigate to Dice Roll 2) Click a number button 3) Observe result 4) Repeat 3 times 5) Check Stats	Result display and stats are being updated correctly for each spin. Cumulative profit is being calculated correctly based on win/loss outcomes after all spins.	Hayden
ST-03-OB	Full Coin Flip session	App running, user profile in localStorage	1) Navigate to Coin Flip 2) Click "Heads" 3) Observe result 4) Click "Tails" 5) Check Stats	The results are properly shown after each flip, the stats are updating live, and the balance field increases/decreases accordingly	Hayden

4.2 Recommendation Workflow

Test ID	Scenario	Preconditions	Steps	Expected Outcome	Assignee
ST-04-OB	Recommendation appears based on friend activity	User A and User B profiles exist, User B has played Slots (Slots in favoriteGames), User A is friends with User B	1) Log in as User A 2) Open Recommendations panel 3) Observe listed games	Slots are included in User A's recommendations with User B as the source friend, and the games already included in User A's favorite list are not included	Adrian
ST-05-OB	Recommendation list updates after adding a new friend	User A has no friends initially, User C has played Coin Flip	1) Log in as User A 2) Check recommendations 3) Add User C as friend 4) Re-open recommendations	With the addition of User C, Coin Flip is included in the recommendations list	Adrian

4.3 Friends List Workflow

Test ID	Scenario	Preconditions	Steps	Expected Outcome	Assignee
ST-06-OB	Add and remove a friend	App running, two user profiles exist in localStorage	1) Navigate to Friends 2) Enter a valid username in the add field 3) Click Add 4) Confirm friend appears in list 5) Click Delete next to that friend	Friend is added to the list after step 3. List is empty again after step 5. Changes persist after page refresh	Hayden
ST-07-OB	Attempt to add a non-existent user	App running, user profile exists	1) Navigate to Friends 2) Type a username that does not exist 3) Click Add	The alert “User does not exist” is triggered. The friends list has not been altered	Hayden

4.4 Account Deletion Workflow (NOT YET EXECUTABLE, Planned for Iteration 3)

Test ID	Scenario	Preconditions	Steps	Expected Outcome	Assignee
ST-08-OB	Delete current account and verify data removal	account.js implemented, user profile exists in localStorage with friends and game history	1) Navigate to Profile/Account 2) Click “Delete Account” 3) Confirm deletion 4) Inspect localStorage	The user is deleted from localStorage. The user is no longer in any remaining user’s friends list. The app navigates away from the profile page	Eric
ST-09-OB	Account deletion removes user from friends’ lists	account.js implemented, User A and User B are mutual friends, User B deletes their account	1) Log in as User B 2) Delete account 3) Log in as User A 4) Check friends list	User B is no longer in User A’s friends list. There are no remaining references	Eric

4.5 Persistence Workflow

Test ID	Scenario	Preconditions	Steps	Expected Outcome	Assignee
ST-10-OB	State persists across page reload	User has played games and added friends	1) Note current stats and friends list 2) Refresh the page (Ctrl+R) 3) Check stats and friends list again	All values are identical after reloading, no data loss from localStorage round-trip	Hayden

5. Demonstration of Coverage

5.1 Test Distribution Summary

Test Level	Clear Box (CB)	Translucent Box (TB)	Opaque Box (OB)	Total
Unit Tests	24	3	0	**27**
Integration Tests	7	5	0	**12**
System Tests	0	0	10	**10**
Total	**31**	**8**	**10**	**49**

5.2 User Story Coverage Mapping

User Story	Covered By	Status
As a user, I want functional game mechanics so I can use the product.	UT-13 through UT-19, IT-01, IT-02, IT-03, ST-01, ST-02, ST-03	**Full Coverage**
As a user, I want to see my statistics to make decisions. / As a user, I want my win/loss ratio so I can brag.	UT-07, UT-08, IT-06, ST-01 through ST-03	**Full Coverage**
As a user, I want to add friends to see who is active.	UT-01 through UT-06; IT-04, IT-05; ST-06, ST-07 IT-10, IT-11, IT-12 cover the Java backend layer only	**Partial - No friend request actions implemented, only dummy accounts**
As a user, I want game recommendations so I don't get bored	UT-20 through UT-23, IT-04, IT-05, ST-04, ST-05	**Full Coverage**
As a user, I want to delete my account for data security.	UT-10, UT-11, UT-12 (TDD Red - planned), ST-08, ST-09 (planned, not yet executable)	**Partial - Stub for TDD, requires account system from iter 3**
As a user, I want my data to persist between sessions	UT-04, UT-05, UT-12, IT-01, IT-02, IT-07, ST-10	**Full Coverage**
As a user, I want to have proper authentication	UT-24 through UT-27, IT-08, IT-09	**Full Coverage**

5.3 High-Risk Components

The following components have the highest risk and thus correspondingly more test coverage:

- Game RNG and profit calculations - incorrect win/loss calculations will destroy all user statistics. Covered by UT-13 to UT-19.
- Account deletion - failure to delete the user from other users' friend lists will result in “orphaned” user data. Covered by UT-10, UT-11, and UT-12, which were all written in TDD's red phase (account.js is currently a stub that throws). ST-08 and ST-09 will be system tests. All five will be executed once account.js is implemented in Iteration 3.
- localStorage - incorrect save or load logic will result in silent data loss for all sessions. Covered by IT-01 to IT-03, IT-07, and ST-10. UT-12 also covers this case but is currently in red phase until account.js is implemented.

- Recommendation algorithm correctness - the recommendation logic needs to correctly count and sort while adhering to the skip own favorites requirement and sorting by popularity. Covered by UT-20 to UT-23.
- Backend session authentication - unauthenticated requests should return 401 before allowing any data operation. Covered by IT-12 and UT-24.

5.4 Known Coverage Gaps & Mitigations

- View layer (DOM): gameView.js, friendsView.js, recView.js are not unit-tested as they require the DOM. This is indirectly tested through the system tests (ST-01 through ST-10).
- JAVA FriendsHandler action logic: the logic of accepting, declining, and canceling the friend requests has not yet been fully implemented. Tests of the critical add and accept friend requests are provided in the tests IT-10 and IT-11. The other paths will be implemented once the implementation is done.
- Concurrent access: the race conditions in SessionManager and UserStore with multiple users are not yet tested, and within the academic scope of the project, will not be considered.
- Network failure handling: frontend behavior when the JAVA server is unreachable, this is not yet considered in the plan.

5.5 Iteration 2 Burndown

Iteration 2 lasted from March 10 to March 23, 2026. All 28 developer days of tracked work were completed, resulting in a new velocity of 28 developer days per iteration, or 80% of our full 35-day team capacity.

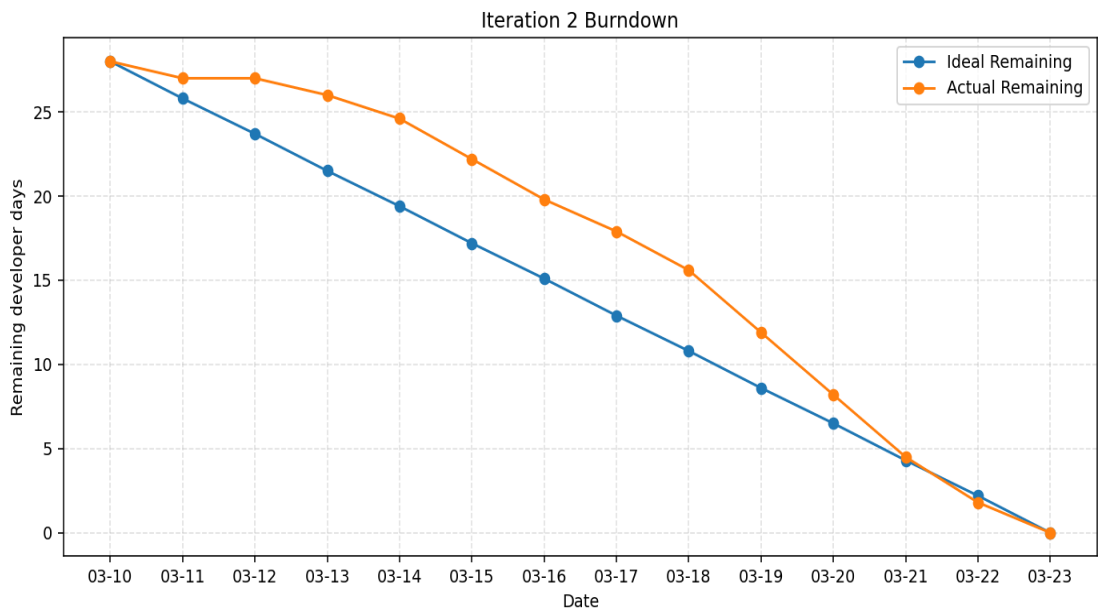

Figure 1: Iteration 2 Burndown Chart (Mar 10 - Mar 23, 2026)